

UNIT-IV Exception Handling and I/O Programming

I/O Programming

Stream Classes in C#

Stream- *The stream is basically the sequence of bytes passing through the communication path.*

There are two main streams: the **input stream** and the **output stream**.

The input stream is used for reading data from file (read operation) and the output stream is used for writing into the file (write operation).

A file is a collection of data stored in a disk with a specific name and a directory path.

When a file is opened for reading or writing, it becomes a stream.

The type of streams includes –

- **Byte Streams** – It includes Stream, FileStream, MemoryStream and BufferedStream.
- **Character Streams** – It includes Textreader-TextWriter, StreamReader, StreamWriter and other streams.

Byte streams have classes that consider data in the stream as byte.

The **System.IO namespace** has various classes that are used for *performing numerous operations with files, such as creating and deleting files, reading from or writing to a file, closing a file etc.*

The stream Classes we discussed are described in the below table –

Sr.No	I/O Class & Description
1	BinaryReader --Reads primitive data from a binary stream.
2	BinaryWriter --Writes primitive data in binary format.
3	BufferedStream --A temporary storage for a stream of bytes.
4	Directory --Helps in manipulating a directory structure.
5	DirectoryInfo --Used for performing operations on directories.
6	DriveInfo --Provides information for the drives.

7	File --Helps in manipulating files.
8	FileInfo --Used for performing operations on files.
9	FileStream --Used to read from and write to any location in a file.
10	MemoryStream --Used for random access to streamed data stored in memory.
11	Path --Performs operations on path information.
12	StreamReader --Used for reading characters from a byte stream.
13	StreamWriter --Is used for writing characters to a stream.
14	StringReader --Is used for reading from a string buffer.
15	StringWriter --Is used for writing into a string buffer.

Console I/O Stream

The Console class from the System namespace helps you with system I/O to the command prompt window or console.

C# Output

In order to output something in C#, we can use

`System.Console.WriteLine()`

OR

`System.Console.Write()`

Here, **System** is a namespace, **Console** is a class within namespace System and WriteLine and Write are methods of class Console.

Difference between WriteLine() and Write() method

The main difference between WriteLine() and Write() is that the Write() method only prints the string provided to it, while the WriteLine() method prints the string and moves to the start of next line as well.

Ex.

```
class Test
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Prints on ");
        Console.WriteLine("New line");
    }
}
```

```
        Console.WriteLine("Prints on ");  
        Console.WriteLine("Same line");  
    }  
}
```

When we run the program, the output will be

Prints on
New line
Prints on Same line

***Combining (Concatenating) two strings using + operator and printing them
Strings can be combined/concatenated using the + operator while printing.***

```
namespace Sample  
{  
    class Test  
    {  
        public static void Main(string[] args)  
        {  
            int val = 55;  
            Console.WriteLine("Hello " + "World");  
            Console.WriteLine("Value = " + val);  
        }  
    }  
}
```

When we run the program, the output will be

Hello World
Value = 55

Printing concatenated string using Formatted String [Better Alternative]

***A better alternative for printing concatenated string is using formatted string.
Formatted string allows programmer to use placeholders for variables.***

For example,

The following line,

Console.WriteLine("Value = " + val);

can be replaced by,

Console.WriteLine("Value = {0}", val);

{0} is the placeholder for variable val which will be replaced by value of val. Since only one variable is used so there is only one placeholder.

Multiple variables can be used in the formatted string. We will see that in the example below.

```
namespace Sample
{
    class Test
    {
        public static void Main(string[] args)
        {
            int firstNumber = 5, secondNumber = 10, result;
            result = firstNumber + secondNumber;
            Console.WriteLine("{0} + {1} = {2}", firstNumber, secondNumber, result);
        }
    }
}
```

When we run the program, the output will be
5 + 10 = 15

Here, {0} is replaced by firstNumber, {1} is replaced by secondNumber and {2} is replaced by result. This approach of printing output is more readable and less error prone than using + operator.

C# Input

*In C#, the simplest method to get input from the user is by using the **ReadLine()** method of the Console class.*

*However, **Read()** and **ReadKey()** are also available for getting input from the user.*

They are also included in **Console class**.

The difference between ReadLine(), Read() and ReadKey() method is:

ReadLine(): *The ReadLine() method reads the next line of input from the standard input stream. It returns the same string.*

Read(): *The Read() method reads the next character from the standard input stream. It returns the ascii value of the character.*

ReadKey(): *The ReadKey() method obtains the next key pressed by user. This method is usually used to hold the screen until user press a key.*

Working With File Class –

C# File Class provides static methods for most of the file operations including create a file, copy and move a file, deleting files, and working with FileStream to read and write streams.

The File class is defined in the System.IO namespace.

Here is a list of the common File class methods.

Sr.No	Method	Description
1	Copy	This method is used to copy a file to the specified location.
2	Create	This method is used to create a file in the specified path.
3	Delete	This method is used to Delete a file.
4	Open	This method is used to return a FileStream object at the specified path.
5	Move	Moves a specified file to a new location. we can specify a different name for the file in the new location.
6	Exists	Determines whether the specified file exists.
7	OpenRead	Opens an existing file for reading.
8	OpenWrite	Opens an existing file or creates a new file for writing.

Create Method

This method is used to create a file in the specified folder.

```
1. File.Create(@"F:\FILEPRO\try.txt");
2. if (File.Exists(@"F:\FILEPRO\try.txt")) ;
3. Console.WriteLine("File is created successfully.....");
```

WriteAllText Method

This method is used to Write Text into a file.

```
4. String rs = "Hello SADSA \n WEIcome to \t GOA";
5. File.WriteAllText(@"F:\FILEPRO\try.txt", rs);
```

Delete Method

This method is used to Delete a file.

```
6. static void Main(string[] args) {
7.     string path = @"F:\FILEPRO\try.txt";
8.     File.Delete(path);
```

```
9.     Console.WriteLine("File has been deleted");
10. }
```

Exists Method

Determines whether the specified file exists.

```
1. static void Main(string[] args) {
2.     string path = @ "D:\MyTestFile1.txt";
3.     Console.WriteLine(File.Exists(path) ? "File exists." : "File does
   not exist.");
4. }
```

Copy Method

This method is used to copy a file to the specified location.

```
1. static void Main(string[] args) {
2.     string path = @ "D:\MyTestFile1.txt";
3.     string path1 = @ "D:\MyTest1.txt";
4.     File.Copy(path, path1);
5.     Console.WriteLine("File has been copied");
6. }
```

Move Method

Moves a specified file to a new location. We can specify a different name for the file in the new location.

```
1. static void Main(string[] args) {
2.     string path = @ "D:\MyTestFile1.txt";
3.     string path1 = @ "c:\MyTest1.txt";
4.     File.Move(path, path1);
5.     Console.WriteLine("File has been moved");
6. }
```

Working With FileInfo Class –

The FileInfo class does not have static methods and can only be used on instantiated objects. The **FileInfo** object represents a file on a disk or network location. It also provides instance methods for the creation, copying, deletion, moving, and opening of files, and aids in the creation of **FileStream** objects. List of Methods of **FileInfo** Class, I have included only a few methods of the **FileInfo Class**.

S.No	Methods	Description
1	Create	This method is used to create the new file.
2	CreateText	This Method Creates a StreamWriter that writes a new text file.
3	Delete	This method is used to delete a existing file.
4	CopyTo	The CopyTo method is used to copy the existing file into new file.
5	MoveTo	The MoveTo method is used to move the file one place to another valid location.
6	AppendText	This method creates a StreamWriter that appends text to the file represented by this instance of the FileInfo.
7	OpenText	This method creates a StreamReader with UTF8 encoding that reads from an existing text file.

Create ()

This method is used to create the new file.

```

1. static void Main(string[] args) {
2.     string path = @"D:\TYC\first.txt";
3.     FileInfo fi = new FileInfo(path);
4.     fi.Create();
5.     { Console.WriteLine(fi.Name + " Created
   Succesfully....."); }
6. }
```

CreateText ()

This Method creates a StreamWriter that writes a new text file.

```

7. FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext1.txt");
8. StreamWriter str = fi.CreateText();
9. str.WriteLine("hello TYECS \n WELCOME in SANGOLA COLLEGE...");
10. Console.WriteLine("File has been created with text");
11. str.Close();
```

Delete()

This method is used to delete an existing file.

```

1. FileInfo fi = new FileInfo(@"D:\MyTestFilecreatetext.txt");
2. fi.Delete();
3. Console.WriteLine("File has been deleted");
```

CopyTo()

The CopyTo method is used to copy an existing file into a new file.

```
1. string path = @ "d:\MyTestFile7.txt";
2. string path2 = @ "d:\NewFile.txt";
3. FileInfo fi1 = new FileInfo(path);
4. FileInfo fi2 = new FileInfo(path2);
5. fi1.CopyTo(path2);
6. Console.WriteLine("{0} was copied to {1}.", path, path2);
```

MoveTo()

The MoveTo method is used to move the file from one place to another valid location.

```
1. string path = @ "d:\NewFile1.txt";
2. string path2 = @ "E:\NewFile1.txt";
3. FileInfo fi1 = new FileInfo(path);
4. FileInfo fi2 = new FileInfo(path2);
5. fi1.MoveTo(path2);
6. Console.WriteLine("{0} was copied to {1}.", path, path2);
```

AppendText()

This method creates a StreamWriter that appends text to the file represented by this instance of the FileInfo.

```
1. FileInfo fi = new FileInfo(@ "E:\NewFile1.txt");
2. StreamWriter sw = fi.AppendText();
3. sw.WriteLine("This");
4. sw.WriteLine("is Extra");
5. sw.WriteLine("Text");
6. Console.WriteLine("File has been appended");
7. sw.Close();
```

OpenText()-

This method creates a StreamReader with UTF8 encoding that reads from an existing text file.

```
1. FileInfo fi = new FileInfo(@ "E:\NewFile1.txt");
2. StreamReader sr = fi.OpenText();
3. string s = "";
4. while ((s = sr.ReadLine()) != null) {
5.     Console.WriteLine(s);
6. }
```


The FileInfo class provides the following properties that enable you to retrieve information about a file.

S.No	Property	Description
1	CreationTime	This property returns the creation of the file date and time.
2	Exists	This property checks for the presence of a file before operating on it.
3	Extension	It returns the type of the file in the extension name
4	FullName	It returns the full name of the file from the root directory.
5	Name	The Name retrieves the name of a file.
6	LastWriteTime	It returns the last file saving date and time
7	Length	The Length retrieves the size of a file.

FileInfo Properties Example

```

1. using System;
2. using System.Collections.Generic;
3. using System.Linq;
4. using System.Text;
5. using System.IO;
6. namespace fileinfoclass1 {
7.     class Program {
8.         static void Main(string[] args) {
9.             FileInfo fi = new FileInfo(@"E:\NewFile1.txt");
10.            Console.WriteLine("File name is {0} ", fi.Name);

            Console.WriteLine("File creation time is {0} ", fi.CreationTime.ToLong
TimeString());
11. Console.WriteLine("File Lastaccesstime is {0} ", fi.LastAccessTime.ToLo
ngDateString());
12. Console.WriteLine("File length is {0} ", fi.Length.ToString
() + " Bytes");
13. Console.WriteLine("File extension is {0} ", fi.Extension);

14. Console.WriteLine("File exist is: ", fi.Exists);
15. Console.WriteLine("File LastWriteTime is {0} ", fi.LastWrit
eTime);
16.     }
17. } }
```

Directory class

System.IO.Directory class in the .NET Framework class library *provides static methods for creating, copying, moving, and deleting directories and subdirectories.*

Before you can use the Directory class, you must import the **System.IO** namespace.

```
1. using System.IO;
```

Create a Folder

Directory.CreateDirectory () method *-creates a directory or folder in the specified path with the specified Windows security.*

You can also create a directory on a remote computer.

The following code snippet creates a Temp folder in C:\ drive if the directory does not exist already. Assuming the account running the code has admin permissions.

```
1. string root = @"C:\Temp";  
2. string subdir = @"C:\Temp\Mahesh";  
3. // If directory does not exist, create it.  
4. if (!Directory.Exists(root))  
5. {  
6.     Directory.CreateDirectory(root);  
7. }
```

Directory.CreateDirectory is also creates a sub directory or sub folder. All you have to do is to specify the path of the folder in which this subdirectory will be created in. The following code snippet creates 'Mahesh' subdirectory in C:\Temp directory.

```
1. // Create sub directory  
2. if (!Directory.Exists(subdir))  
3. {  
4.     Directory.CreateDirectory(subdir);  
5. }
```

Delete a folder

Directory.Delete() method *deletes an empty folder from the specified path permanently.*

If a folder has sub folders and/or files, you must delete them before you can delete a folder.

If you try to delete a file that is not empty, you will get an error message.

```
1. string root = @"C:\Temp";
2. // If directory does not exist, don't even try
3. if (Directory.Exists(root))
4. {
5.     Directory.Delete(root);
6. }
```

The following code snippet checks if a directory has subdirectories and files and delete them before deleting a directory.

Check if a folder Exists

Directory.Exists method checks if the specified directory exists.

The following code snippet checks if a directory exists or not and deletes only if the directory exists.

```
1. string root = @"C:\Temp";
2. // If directory does not exist, don't even try
3. if (Directory.Exists(root))
4. {
5.     Directory.Delete(root);
6. }
```

Move a folder

Directory.Move method moves an existing directory to a new specified directory with full path.

The Move method takes two parameters. The Move method deletes the original directory.

The following code snippet moves the source directory to the destination directory.

```
1. string sourceDirName = @"C:\Temp";
2. string destDirName = @"C:\NewTemp";
3. try
4. {
5.     Directory.Move(sourceDirName, destDirName);
6. }
7. catch (IOException exp)
```

```

8. {
9.     Console.WriteLine(exp.Message);
10. }

```

C# DirectoryInfo Class

The DirectoryInfo class shares almost all of the same properties as the FileInfo class, except that they operate on directories not files.

The DirectoryInfo class does not have static methods and can only be used on instantiated objects.

The DirectoryInfo object represents a physical directory on a disk.

It also provides instance methods for deleting folders, creating a new directory and sub directories.

When to use C# DirectoryInfo class-

If you are going to reuse an object several times, consider using the instance method of DirectoryInfo instead of the corresponding static methods of the Directory class.

Some of the most useful methods of the DirectoryInfo class are:

S.No	Methods	Description
1	Create	This method is used to create the new directory.
2	CreateSubdirectory	This method is used to create a subdirectory or subdirectories on the specified path.
3	MoveTo	Moves a DirectoryInfo instance and its contents to a new path.
4	Delete	Deletes this instance of a DirectoryInfo, specifying whether to delete subdirectories and files.
5	GetDirectories	This method is used to get the sub directories in the given directory path.
6	GetFiles	The GetFiles method is used to get the files in the specified folder.

Create a new directory in C#

The Create method is used to create the new directory.

```

1. static void Main(string[] args)
1. {    DirectoryInfo di = new DirectoryInfo(@"D:\TRIAL");

```

```
2.     di.Create();
3.     { Console.WriteLine(di.Name + " Created Successfully...."); }
4. }
```

Create SubDirectory in C#

The CreateSubdirectory method is used to create a subdirectory or subdirectories on the specified path.

```
5. static void Main(string[] args)
1. {
2.     DirectoryInfo di = new DirectoryInfo(@"D:\TRIAL");
3.     di.CreateSubdirectory(@"SUB1");
4.     { Console.WriteLine( " Sub Directory Created
        Successfully...."); }
```

Move a directory in C#

The MoveTo method of moves a directory to a new location including its contents.

```
5. static void Main(string[] args)
6. {
7.     String src = @"D:\TY";
8.     string dest = @"D:\SY";
9.     DirectoryInfo f1 = new DirectoryInfo(src);
10.    DirectoryInfo f2 = new DirectoryInfo(dest);
11.    f1.MoveTo(path1);
12.    {
13.        Console.WriteLine("Directory has been Moved");
14.    }
15. }
```

Delete a directory in C#

The Delete method deletes a directory, specifying whether to delete subdirectories and files.

```
1. static void Main(string[] args) {
2.     string path = @ "D:\TY";
3.     DirectoryInfo f1 = new DirectoryInfo(path);
4.     f1.Delete(); {
5.     Console.WriteLine("Directory has been deleted");
6. }
7. }
```

Get Subdirectories of a directory in C#

GetDirectories method is used to get sub directories in the given directory path, or returns the subdirectories of the current directory.

```

1. static void Main(string[] args) {
2.     try {
3.         DirectoryInfo[] alsub = di.GetDirectories();
4.         Console.WriteLine("The number of directories containing is
           "+alsub.Length);
5.         for (int i = 0; i < alsub.Length; i++)
6.             Console.WriteLine(" " + alsub[i].Name);
7.     }
8.     catch (Exception e) {
9.         Console.WriteLine("The process failed: {0}", e.ToString());
10.    }
11. }
```

Get all files in a folder in C#

The GetFiles method is used to get the files in the specified folder. The following code loops through all directories and gets sub directories and loops through them to get all files.

```

10. static void Main(string[] args)
11. {
12.     FileInfo [] alfiles = di.GetFiles();
13.     for (int i = 0; i < alfiles.Length; i++)
14.         Console.WriteLine(" " + alfiles[i].Name);
15. }
```

Properties Of DirectoryInfo Class-

The DirectoryInfo class provides the following properties that enable you to retrieve information about a directory.

S.No	Property	Description
1	CreationTime	This property returns the creation of the directory date and time.
2	Exists	It returns whether the directory exists or not as Boolean result.
3	FullName	It returns the full name of the file from the root directory.
4	Name	It returns the name of the directory
5	LastAccessTime	It returns the date and time of the last access time
6	LastWriteTime	It returns the last file saving date and time
7	Root	Gets the root portion of a path.

DirectoryInfo Properties example.

```
1. using System;
2. using System.IO;
3. namespace directoryinfoproperty {
4.     class Program {
5.         static void Main(string[] args) {
6.             DirectoryInfo fi = new DirectoryInfo(@"D:\newfile");
7.             Console.WriteLine("Directory name is {0} ", fi.Name);
8.             Console.WriteLine("Directory creation time is {0} ", fi.CreationTime.ToLongTimeString());
9.             Console.WriteLine("Directory Lastaccesstime is {0} ", fi.LastAccessTime.ToLongDateString());
10.            Console.WriteLine("Directory exist is: ", fi.Exists);
11.            Console.WriteLine("Directory LastWriteTime is {0} ", fi.LastWriteTime);
12.            Console.WriteLine("Directory root is {0} ", fi.Root);
13.        }
14.    }
15. }
```